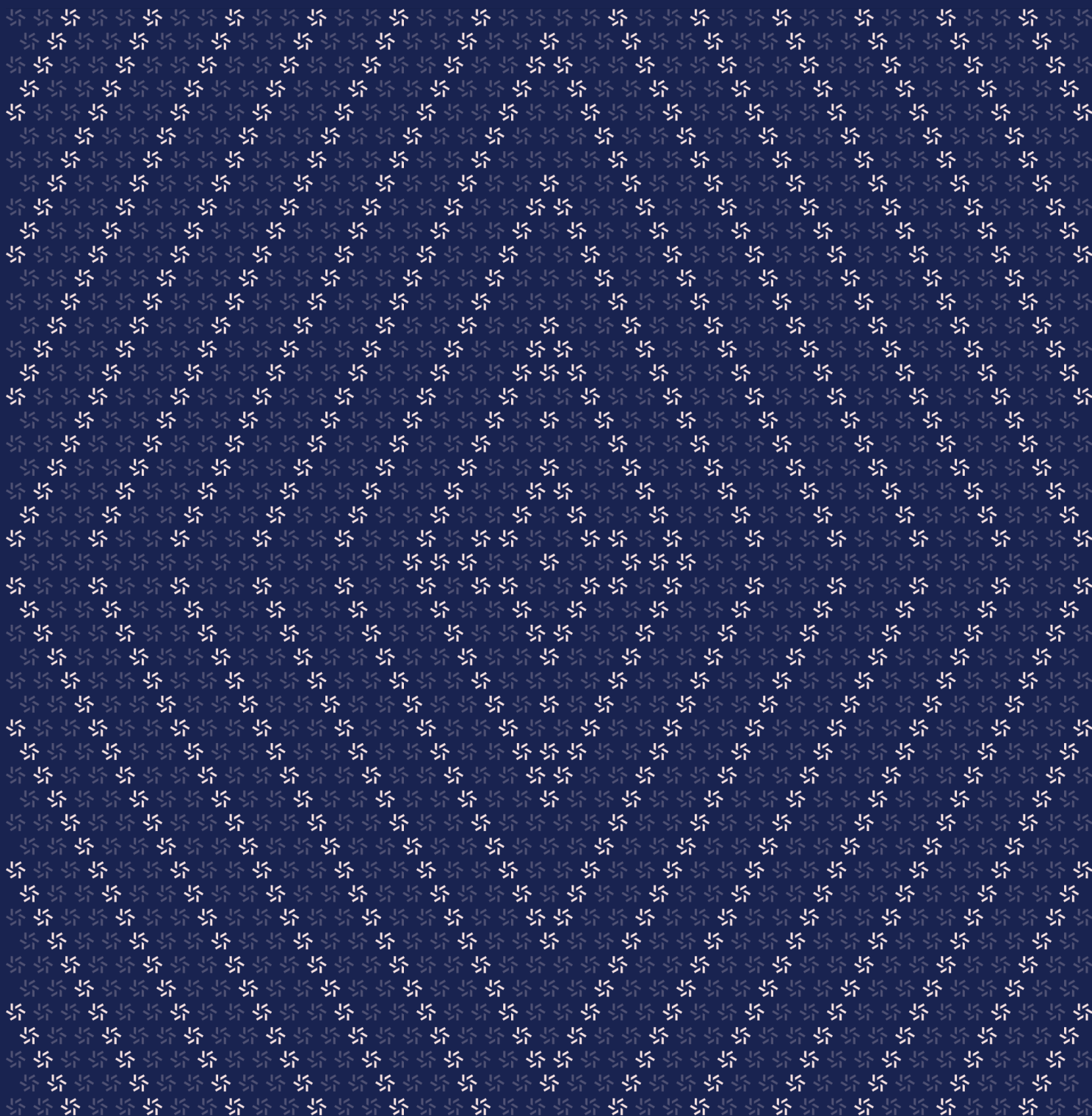


August 18, 2026

EVM Deposit/Withdraw Bridge

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About EVM Deposit/Withdraw Bridge	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. Gas griefing during native withdrawal settlement	11
3.2. Token recovery everSupported restriction does not limit owner behavior	13
3.3. Withdrawer gas limits may cause incorrect withdrawal failures	15
3.4. Asset-keyed storage layout gas optimization	17
3.5. Reentrancy guard is not upgradable	19

4.	Discussion	20
4.1.	A single withdrawal nonce across multiple withdrawers could result in coordination hazards	20
5.	System Design	22
5.1.	Component: PcfaBridge	22
6.	Assessment Results	24
6.1.	Disclaimer	24

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Pacifica on August 17th, 2026. During this engagement, Zellic reviewed EVM Deposit/Withdraw Bridge's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any situations in which a nonce could be replayed in the emitted events?
 - Can unpermissioned actors withdraw assets from the bridge?
 - Can a malicious depositor cause the contract to emit a `Deposit` event with false information that does not correspond to the assets they actually deposited?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped EVM Deposit/Withdraw Bridge contracts, we discovered five findings. No critical issues were found. One finding was of medium impact and the other findings were informational in nature.

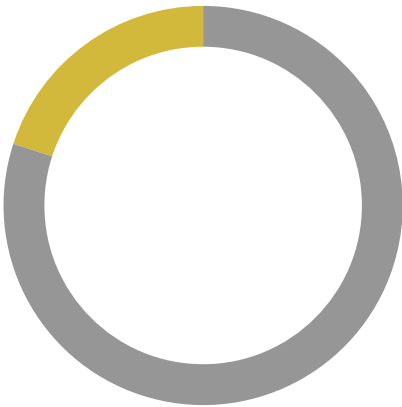
Pacifica confirmed that the deployed codebase corresponds to the frozen state at commit `2c3338d1894d407456bd00f466390d828393c0be`, with no code changes made between that commit and deployment.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of

Pacifica in the Discussion section ([4](#), [7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	0
Informational	4



2. Introduction

2.1. About EVM Deposit/Withdraw Bridge

Pacifica contributed the following description of EVM Deposit/Withdraw Bridge:

Pacifica is a decentralized perp exchange supporting a wide range of assets, including crypto, RWA and exotic options. The audit specifically checks the EVM deposit/withdrawal bridge contract for the exchange.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

EVM Deposit/Withdraw Bridge Contracts

Type	Solidity
Platform	EVM-compatible
Target	bridge-contract
Repository	https://github.com/pacifica-fi/bridge-contract ↗
Version	950957c3d4d936b79b9f6629ce6a6c7facb9e6d0
Programs	PcfaBridge.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of two person-days. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
 ↗ Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Kuilin Li
 ↗ Engineer
kuilin@zellic.io ↗

Ayaz Mammadov
 ↗ Engineer
ayaz@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

August 17, 2026 Start of primary review period

August 17, 2026 End of primary review period

3. Detailed Findings

3.1. Gas grieving during native withdrawal settlement

Target	PcfaBridge.sol		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

During the native value withdrawal process, the recipients are called in a loop, where each recipient receives a call:

```
function _settleNativeWithdrawals(
    BridgeLane lane,
    address asset,
    uint64 nonce,
    WithdrawalRecord[] calldata withdrawals,
    WithdrawalRecord[] memory settledWithdrawals
) private returns (uint256 settledCount) {
    // [...]
    for (uint256 index = 0; index < withdrawals.length; ++index) {
        WithdrawalRecord calldata withdrawal = withdrawals[index];
        (bool succeeded,) = withdrawal.recipient.call{value:
            withdrawal.amount}("");
        if (succeeded) {
            // [...]
        }
    }
}
```

This low-level call does not specify a gas limit, so the maximum possible gas (63/64 of the remaining gas) is forwarded into the call. The recipient contract can therefore consume as much of that gas as it is able to.

Because only 1/64 of the pre-call gas remains afterward, the execution context is unlikely to have enough gas left to run the remainder of the logic.

Impact

A malicious recipient can block other recipients from receiving their funds by consuming all the gas.

It is also theoretically possible for a malicious recipient to collude with the block builder to make the sender of the withdrawal transaction spend more gas. This increases the value of the transaction to the block builder, and part of that value can be captured back to the recipient,

incentivizing such an attack.

Recommendations

Enforce a gas limit when calling recipients.

Remediation

This issue has been acknowledged by Pacifica, and a fix was implemented in commit [2c3338d1](#) ↗.

3.2. Token recovery everSupported restriction does not limit owner behavior

Target	PcfaBridge.sol		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `recoverUnsupportedToken` function is for the recovery of ERC-20 tokens that were incorrectly sent to the bridge:

```
/// @notice Recovers an ERC-20 that has never been configured as a supported
/// asset in either lane.
/// @dev Not gated on pause state; the `everSupported` check is the only
/// guard, so this can never move
/// funds held for an asset the bridge has enabled for deposits or withdrawals.
// [...]
function recoverUnsupportedToken(address asset, address recipient,
    uint256 amount) external onlyOwner nonReentrant {
    // [...]
    if (!_bridgeStorage().everSupported[asset])
        revert SupportedAssetRecovery(asset);
    IERC20(asset).safeTransfer(recipient, amount);
    emit UnsupportedTokenRecovered(asset, recipient, amount);
}
```

As the comment notes, the `everSupported` check prevents this recovery function from moving funds held for an asset that the bridge has ever enabled. The function is also `onlyOwner`, so only the owner can recover assets.

However, the contract is upgradable, and upgrade authorization is `onlyOwner` as well:

```
function _authorizeUpgrade(address) internal override onlyOwner {}
```

This means that the owner has effective full control over all the assets owned by this contract, because they can always upgrade the contract to an implementation that transfers the assets.

Impact

The `everSupported` guard overstates what the contract guarantees. An owner can still transfer assets that the bridge has previously enabled, either through a contract upgrade that clears the `everSupported` flag or through a contract upgrade that transfers the assets directly.

Recommendations

We recommend adding a lower-privileged role so that the account that assists with token recovery can be meaningfully restricted by the `everSupported` flag.

Alternatively, with no code changes, we recommend documenting that the `everSupported` flag is only a guardrail and not a security guarantee, since the owner is permitted to upgrade the contract.

Note that it is not feasible to determine, during a contract upgrade, whether a future version of the contract would respect the same invariant.

Remediation

This issue has been acknowledged by Pacifica, and a fix was implemented in commit [2c3338d1](#).

3.3. Withdrawer gas limits may cause incorrect withdrawal failures

Target	PcfaBridge.sol		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

During the token value withdrawal process, the contract calls itself using a try-catch in order to catch reverts during the withdrawal process:

```
function _settleTokenWithdrawals(
    // [...]
) private returns (uint256 settledCount) {
    for (uint256 index = 0; index < withdrawals.length; ++index) {
        WithdrawalRecord calldata withdrawal = withdrawals[index];
        try this.withdrawTokenRecord(asset, withdrawal.recipient,
            withdrawal.amount) {
            // [...]
        } catch (bytes memory reason) {
            _emitWithdrawalFailure(lane, asset, nonce, index, withdrawal,
                reason);
        }
    }
}
```

The withdrawTokenRecord function then makes another call to transfer the token:

```
function withdrawTokenRecord(address asset, address recipient, uint256 amount)
    external {
    if (msg.sender != address(this)) revert UnauthorizedSelfCall();
    // [...]
    token.safeTransfer(recipient, amount);
    // [...]
}
```

The intent of the try-catch is to allow other withdrawals to proceed even if one of them fails due to a downstream issue with the specific token implementation contract.

However, the withdrawTokenRecord function may revert through no fault of the token implementation contract if it was not given enough gas to complete. The amount of gas available

to it is ultimately set by the caller of `batchWithdraw` or `spotBatchWithdraw`.

Impact

Note that the sender of the `batchWithdraw` or `spotBatchWithdraw` functions is a permissioned withdrawer with a role, so they are broadly trusted to withdraw assets properly.

However, if they call those functions with a carefully limited quantity of gas, then it is possible for them to force a revert inside the child execution context. Depending on optimization parameters and gas costs, there may be enough gas left over to complete the parent context. If it does complete, then the sender will have manufactured a withdrawal failure.

Recommendations

We recommend measuring the amount of gas left before the call to `withdrawTokenRecord` in order to enforce a gas minimum. If the call succeeds, then the gas does not need to be checked. However, if the call fails and the minimum was violated, then it should be treated as a revert of the entire call to `batchWithdraw` or `spotBatchWithdraw` rather than as a single withdrawal failure.

Note that although the gas does not strictly need to be checked if the call succeeds, it may be more friendly to gas estimation algorithms to check the gas left unconditionally.

Remediation

This issue has been acknowledged by Pacifica, and a fix was implemented in commit [2c3338d1](#).

3.4. Asset-keyed storage layout gas optimization

Target	PcfaBridge.sol		
Category	Optimization	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The PcfaBridge contract uses an ERC-7201 storage struct for all of its storage requirements:

```
struct BridgeStorage {
    mapping(address asset => AssetStatus status) balanceAssets;
    mapping(address asset => AssetStatus status) spotAssets;
    mapping(address asset => bool) everSupported;
    mapping(address asset => uint64 nonce) balanceWithdrawNonces;
    mapping(address asset => uint64 nonce) spotWithdrawNonces;
    mapping(address asset => uint64 nonce) balanceDepositNonces;
    mapping(address asset => uint64 nonce) spotDepositNonces;
    uint64 maxBatchSize;
    bool withdrawalsPaused;
}
```

Each call to a function on the bridge can read and write to multiple mappings in this storage struct, all keyed by asset, which is the address of the token contract.

Per the [Solidity storage layout](#) interface, the slots of each item in each of these mappings is determined by hashing a concatenation of a hash of the mapping key (in this case, the asset) with the slot. This means that the asset needs to be hashed multiple times in order to access each of these fields.

Most actions, however, use multiple fields. For instance, a single call to deposit accesses the balanceAssets field to determine whether the asset is depositEnabled and then accesses the balanceDepositNonces field to read and set the relevant nonce.

So, the asset is being hashed multiple times with this storage layout.

Impact

This storage layout causes unnecessary hashing, due to the repetitive asset key.

Recommendations

We recommend adding a single `AssetData` struct definition and storing all of the current fields in a single mapping keyed by asset address:

```
struct AssetData {
    AssetStatus balanceAsset;
    AssetStatus spotAsset;
    bool everSupported;
    uint64 balanceWithdrawNonce;
    uint64 spotWithdrawNonce;
    uint64 balanceDepositNonce;
    uint64 spotDepositNonce;
}
struct BridgeStorage {
    mapping(address asset => AssetData data) assetData;
    uint64 maxBatchSize;
    bool withdrawalsPaused;
}
```

With this or a similar storage layout, each asset address would need to be hashed only once in order to obtain the `AssetData` storage pointer, and all subsequent storage reads and writes could be based on fixed offsets from that pointer.

Remediation

This issue has been acknowledged by Pacifica, and a fix was implemented in commit [2c3338d1](#).

3.5. Reentrancy guard is not upgradable

Target	PcfaBridge.sol		
Category	Optimization	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The PcfaBridge contract imports OpenZeppelin's upgradable proxy utilities alongside the non-upgradable ReentrancyGuard module:

```
import {Initializable}
    from "@openzeppelin-contracts/proxy/utils/Initializable.sol";
import {UUPSUpgradeable}
    from "@openzeppelin-contracts/proxy/utils/UUPSUpgradeable.sol";
import {ReentrancyGuard}
    from "@openzeppelin-contracts/contracts/ReentrancyGuard.sol";
```

The ReentrancyGuard module from OpenZeppelin is not upgradable.

Impact

If storage slots change unexpectedly during a future upgrade, the reentrancy guard or other logic sharing the same storage slot could break.

Recommendations

Use ReentrancyGuardTransient, which uses transient storage to reduce gas costs and avoids consuming persistent storage slots for the reentrancy guard, making it well-suited to upgradable contracts.

Remediation

This issue has been acknowledged by Pacifica, and a fix was implemented in commit [2c3338d1](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. A single withdrawal nonce across multiple withdrawers could result in coordination hazards

Every EVM account that holds the withdrawer role for a specific lane-asset pair can call the `batchWithdraw` and `spotBatchWithdraw` functions to withdraw assets:

```
/// @inheritdoc IPcfaBridge
function batchWithdraw(address asset, uint64 nonce, WithdrawalRecord[]
    calldata withdrawals)
    external
    nonReentrant
    whenNotPaused
{
    _checkRole(withdrawerRole(BridgeLane.Balance, asset), msg.sender);
    _batchWithdraw(BridgeLane.Balance, asset, nonce, withdrawals);
}

/// @inheritdoc IPcfaBridge
function spotBatchWithdraw(address asset, uint64 nonce, WithdrawalRecord[]
    calldata withdrawals)
    external
    nonReentrant
    whenNotPaused
{
    _checkRole(withdrawerRole(BridgeLane.Spot, asset), msg.sender);
    _batchWithdraw(BridgeLane.Spot, asset, nonce, withdrawals);
}
```

The nonce parameter in these calls refers to a nonce that is keyed by the specific lane-asset pair, which is then incremented after use:

```
function _batchWithdraw(
    BridgeLane lane,
    address asset,
    uint64 suppliedNonce,
    WithdrawalRecord[] calldata withdrawals
) private {
    // [...]
    uint64 expectedNonce = _withdrawNonce(bridgeStorage, lane, asset);
```

```
if (suppliedNonce != expectedNonce) revert UnexpectedWithdrawalNonce();  
// [...]  
_setWithdrawNonce(bridgeStorage, lane, asset, expectedNonce + 1);  
// [...]  
}
```

Because this nonce is only keyed by the specific lane-asset pair, it is not keyed by the withdrawer address. This means that all of the accounts that do withdrawals on a given lane-asset pair have to share the nonce parameter. One successful withdrawal by one withdrawer will affect the behavior of all the other withdrawers because the nonce increased by one.

For a single EVM account, the EVM nonce field in the transaction enforces an order in which pending transactions can be confirmed in a block. Once there are multiple EVM accounts, however, this guarantee can no longer be enforced. The withdrawers have to coordinate among themselves to ensure that each nonce is used only once, and, if any EVM block contains multiple valid withdrawals, those withdrawals could be reordered by the block builder so that only one of them succeeds.

Unlike an out-of-order EVM account nonce, an out-of-order nonce field on a transaction sent by a different withdrawer is still eligible for inclusion in the block. The transaction would simply revert, which means that the gas price is still paid and the EVM nonce is still consumed. The transaction must then be re-signed and resubmitted later.

If these effects are undesirable, then we recommend reviewing the purpose of this nonce field and updating the design accordingly. One alternative design would be to allow nonces to be consumed in any order.

This issue has been acknowledged by Pacifica. Pacifica noted that there will only ever be at most one withdrawer per specific lane-asset pair.

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: PcfaBridge

Description

The PCFA bridge contract manages a portion of the customer funds entrusted to Pacifica on the Monad EVM-compatible chain. It has a centralized owner, which, per documentation, must be a multi-signature wallet. This owner manages the asset whitelist, the withdrawer roles, and the pause/unpause guardian roles, and it can also upgrade the contract.

Attack surface

Any user can deposit assets or native value to the contract using the `deposit` and `spotDeposit` functions, which deposit the asset using the `Balance` and `Spot` lanes, respectively.

Only the `deposit` and `spotDeposit` functions were reviewed as the formal attack-surface scope, because they are the only functions that do not require a specifically assigned and trusted role. The other functions were reviewed in the context of their higher-privilege callers.

Invariants

For the `deposit` and `spotDeposit` functions, the `lane` and `asset` parameters must match a whitelisted combination, and the amount of the asset is actually transferred. The depositor can, however, specify an arbitrary `recipient` address and arbitrary `memo` data. The net on-chain effect of a deposit is the assignment of a new deposit nonce and the emission of a `Deposit` event.

The `batchWithdraw` and `spotBatchWithdraw` functions require the `withdrawer` role for the corresponding lane-asset pair. Anyone holding that role can withdraw any amount of those assets from the bridge. Withdrawal records must be specified, but they do not undergo further balance or privilege checks. Each specific withdrawal either completes or emits a failure event.

In the event of a compromised, malfunctioning, or malicious asset implementation, the `withdrawer` should still be able to submit withdrawals that attempt to move the impacted assets. These withdrawals should not affect other withdrawals beyond the consumption of gas.

Conversely, in the event of a malfunctioning `withdrawer`, the `withdrawer` should not be able to fabricate a failed withdrawal. Finding [3.3](#) discusses an edge case in which this invariant was violated.

The other functions implemented by this contract, such as `setBalanceAsset`, `setSpotAsset`, and `setGuardian`, are configuration functions that are callable only by a guardian or the owner. The guardians and the owner are assumed to be absolutely trusted.

Additionally, due to the balance checks in `_receiveExactToken` and `withdrawTokenRecord`, this contract does not support fee-on-transfer or rebasing tokens. Every transfer is checked, and, if a transfer results in an unexpected amount of value being deducted or credited, the transfer is reverted and treated as a failure.

Test coverage

The tests were reviewed for coverage of the `deposit` and `spotDeposit` functions.

Cases covered

- Token-asset `deposit` and `spotDeposit` work as intended, moving the relevant assets and emitting the `Deposit` event correctly.
- Native-asset `deposit` and `spotDeposit` work as intended, moving the relevant assets and emitting the `Deposit` event correctly.
- Deposits referencing unknown lane-asset pairs revert.
- Deposits during protocol-level pauses revert.
- Deposits of fee-on-transfer tokens, when detectable, revert.

Cases not covered

None found.

6. Assessment Results

During our assessment on the scoped EVM Deposit/Withdraw Bridge contracts, we discovered five findings. No critical issues were found. One finding was of medium impact and the other findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.